Circles-CMG Audit

Gnosis Ltd - Report by Côme du Crest

2024-04-01

Table of contents

- Table of contents
- Circles-CMG Audit
  - Scope
  - Context
  - Status
- Issues
  - [High] Anyone can send worthless pCRC to CMG's vault to break it
  - [Low] Anyone can empty tracking of collateral in CMGRedemptionHandler
  - [Low] CMGRedemptionHandler may be incorrect when adding / removing a collateral twice
  - [Info] Trust and collateral tracking may be incorrect when turned off and on again
  - Optimisations and miscellaneous

Circles-CMG Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Gnosis Ltd.

Scope

The scope includes all contracts within `aboutcircles/circles-groups` at tag `v1.0.0-cmg-prerelease` excluding the deprecated contracts within `src/core-members-group/deprecated-helpers`. The last commit of the tag available at the time is `0x62308de`.

Context

Core member groups are CirclesV2 groups that allows different humans to pool their personal circles (pCRC) into shared group circles (gCRC). The goal is to create a liquid currency backed by a number of different CRC.

The `CoreMembersGroup` represent the address that will be registered as a group on the CirclesV2 Hub. The owner of the group can set the addresses (pCRC) trusted by the group. The CMG can have an optional `CMGRedemptionHandler` which is called when collateral is added/removed to track the balances of pCRC held by the group. The redemption handler exposes functions to figure what collateral can be received when redeeming gCRC.

The CMG can also have an optional `CMGMintHandler` which mimics the trust from the group to other CRCs. The mint handler allows users to send pCRC tokens to it in exchange for gCRC. The mint handler will automatically call `hub.groupMint(CMG, ...)` when receiving pCRC and send the minted gCRC back to the original sender.

The last core contract available is `GroupLiquidityProvider` which can hold gCRC sent by its owner or pCRC that it gets through its `rebalance()` function that redeem gCRC into pCRC. The goal is to make the gCRC tokens liquid.

Contracts interactions are as follow: - Owner of CMG sets trust from CMG to members - Anyone can call `hub.groupMint(CMG, collaterals, amounts, data)` to mint group tokens by sending trusted collateral. This calls `beforeMintPolicy()` on the registered `mintPolicies[CMG]` from the hub. The registered mint policy is the `CoreMembersGroup` contract itself. It registers the mint to `CMGRedemptionHandler` and simply accept the mint. The only requirement is thus that the group trusts the input CRC tokens. This sends the collateral CRC to the treasury of the CMG which is the `StandardTreasury` contract. This treasury transfers the CRC to the `StandardVault` contract for the CMG. - Alternatively, users can send their tokens to the `CMGMintHandler` that

will trigger `onERC1155Received()` which calls `hub.groupMint()` for the user and transfer the received gCRC tokens back to the user. - To redeem group circles, users can send their gCRC back to the `StandardTreasury` which will trigger `onERC1155Received()` on the treasury. It will then call `beforeRedeemPolicy()` on the registered `mintPolicies[CMG]` from the hub. The registered policy is the CMG which registers the redemption to `CMGRedemptionHandler`. The treasury then asks the vault to return the collateral back to the user. There is no security guarantee enforced here apart from the fact that gCRC tokens need to be spent and pCRC are sent back by the treasury. - The `GroupLiquidityProvider` trusts the gCRC tokens and approves the `CMGRedemptionOperator` for all tokens. Its owner deposits gCRC and/or pCRC into it. - Anyone can call `GroupLiquidityProvider` to `rebalance()` its tokens. This will call `redemptionOperator.redeemWithFoundCollateral()`. The redemption operator calls the redemption handler of the CMG to find what collateral is available for redemption. It will then send the gCRC held by `GroupLiquidityProvider` to the group's `StandardTreasury` with encoded values to receive underlying pCRC collateral. The standard treasury will then ask the group's vault to return the encoded collateral to the `GroupLiquidityProvider`. - Anyone can use `GroupLiquidityProvider` in a circles path using gCRC because it trusts gCRC and holds pCRC after rebalance.

Status

The report has been sent to the core developer.

Issues

[High] Anyone can send worthless pCRC to CMG's vault to break it

Summary

The external function `CMGRedemptionHandler.syncValidCollateral()` takes as input collateral IDs and queries the balances of the vault on the hub for these IDs. It then tracks IDs with sufficient balances as valid collateral. Anyone can send worthless unconnected CRC to the vault and call `syncValidCollateral()` with their IDs to propose them as valid collateral.

When `CMGRedemptionOperator.redeemWithFoundCollateral()` is called, it will find the worthless CRC's ID when calling `redemptionHandler.findCollateral()` and burn gCRC in exchange for these worthless pCRC. This will result in broken accounting where one gCRC is backed by more than one pCRC and destroy the value of gCRC.

Vulnerability Detail

Anyone can send worthless CRC tokens to a `StandardVault` as it has no requirements on the tokens it receives:

```
    function onERC1155Received(
        address,
        address,
        uint256,
        uint256,
        bytes memory
    ) public virtual override returns (bytes4) {
        return this.onERC1155Received.selector;
    }
```

The function `CMGRedemptionHandler.syncValidCollateral()` syncs the collateral available on the group vault with no regards to what the collateral is:

```
    function syncValidCollateral(uint256[] memory _collateralIds) external {
        address vault = _getGroupVault();

        // expand addresses array for batch balance check
        address[] memory accounts = new address[](_collateralIds.length);
        for (uint256 i = 0; i < _collateralIds.length; i++) {
            accounts[i] = vault;
        }

        // get balances for all IDs in single call
        uint256[] memory balances = circlesCore.hub.balanceOfBatch(accounts,
            _collateralIds);

```

```
13          // update tracking for each ID based on balance
14          for (uint256 i = 0; i < _collateralIds.length; i++) {
15              uint256 id = _collateralIds[i];
16              uint256 balance = balances[i];
17
18              // add to active tracking if has balance above minimal amount but
                    not tracked
19              if (balance > minimalTrackingAmount) {
20                  // no-op if already tracked
21                  _addActiveId(id);
22              }
23              // remove from active tracking if has balance below minimal amount
                    but is tracked
24              else if (balance <= minimalTrackingAmount) {
25                  // no-op if id not tracked - should not occur
26                  _removeActiveId(id);
27              }
28          }
29      }
```

The function `CMGRedemptionOperator.redeemWithFoundCollateral()` will query `redemptionHandler.findCollateral()` which will return the worthless CRC now tracked by the redemption handler and redeem gCRC for worthless CRC by sending the gCRC to the treasury:

```
1       function redeemWithFoundCollateral(address _group, uint256 _amountToRedeem,
            bool _partialFillable)
2           external
3           nonReentrant
4       {
5           ICMGRedemptionHandler redemptionHandler = ICMGRedemptionHandler(
                _assertGroupAndRedemptionHandler(_group));
6
7           // find collateral using groups redemption handler
8           (uint256[] memory collateralIds, uint256[] memory amounts) =
9               redemptionHandler.findCollateral(_group, _amountToRedeem,
                    _partialFillable);
10
11          uint256 totalValueFound = 0;
12          for (uint256 i = 0; i < amounts.length; i++) {
13              totalValueFound += amounts[i];
14          }
15          ...
16
17          // rely on the redemption handler to structure the data in case other
                formats are adopted
18          bytes memory data = redemptionHandler.structureRedemptionData(
                collateralIds, amounts);
19
20          // to redeem the group Circles must be sent to StandardTreasury with
                the correct data formatted.
21          circlesCore.hub.safeTransferFrom(
22              msg.sender, address(circlesCore.standardTreasury), _toTokenId(
                    _group), totalValueFound, data
23          );
```

```
        ...
    }
```

The `StandardTreasury` will call `_redeemGroupCircles` which will call `beforeRedeemPolicy()` on the CMG with no condition on the collateral IDs to redeem:

```
    function onERC1155Received(address _operator, address _from, uint256 _id,
        uint256 _value, bytes calldata _data)
        public
        override
        onlyHub(0xE8)
        returns (bytes4)
    {
        (bytes32 metadataType, address group, bytes memory userData) =
            _decodeMetadataForGroup(_data);
        if (metadataType == METADATATYPE_GROUPMINT) {
            return _lockCollateralGroupCircles(_id, _value, group, userData);
        } else if (metadataType == METADATATYPE_GROUPREDEEM) {
            return _redeemGroupCircles(_operator, _from, _id, _value, userData)
                ;
        } else {
            // Treasury: Invalid metadata type for received
            revert CirclesStandardTreasuryInvalidMetadataType(metadataType, 0);
        }
    }

    ...

    function _redeemGroupCircles(address _operator, address _from, uint256 _id,
         uint256 _value, bytes memory _userData)
        internal
        returns (bytes4)
    {
        address group = _validateCirclesIdToGroup(_id);
        IStandardVault vault = vaults[group];
        ...

        // query the hub for the mint policy
        IMintPolicy policy = IMintPolicy(hub.mintPolicies(group));
        ...

        // query the mint policy for the redemption values
        uint256[] memory redemptionIds;
        uint256[] memory redemptionValues;
        uint256[] memory burnIds;
        uint256[] memory burnValues;
        (redemptionIds, redemptionValues, burnIds, burnValues) =
            policy.beforeRedeemPolicy(_operator, _from, group, _value,
                _userData);

        ...
        // return collateral Circles to the redeemer of group Circles
        vault.returnCollateral(_from, redemptionIds, redemptionValues,
            _userData);
        ...
```

```
44      }
```

The `CMG.beforeRedeemPolicy()` does not enforce any control on the redeemed collateral IDs:

```
1       function beforeRedeemPolicy(
2           address, /*_operator*/
3           address, /*_redeemer*/
4           address, /*_group*/
5           uint256, /*_value*/
6           bytes calldata _data
7       )
8           external
9           override
10          onlyHubOrTreasury
11          returns (
12              uint256[] memory _ids,
13              uint256[] memory _values,
14              uint256[] memory _burnIds,
15              uint256[] memory _burnValues
16          )
17      {
18          // simplest policy is to return the collateral as the caller requests
                it in data
19          BaseMintPolicyDefinitions.BaseRedemptionPolicy memory redemption =
20              abi.decode(_data, (BaseMintPolicyDefinitions.BaseRedemptionPolicy))
                    ;
21
22          // and no collateral gets burnt upon redemption
23          _burnIds = new uint256[](0);
24          _burnValues = new uint256[](0);
25
26          // register redemption with redemption handler
27          if (redemptionHandlerConnected) {
28              _registerRedemption(redemption.redemptionIds, redemption.
                    redemptionValues);
29          }
30
31          // standard treasury checks whether the total sums add up to the amount
                 of group Circles redeemed
32          // so we can simply decode and pass the request back to treasury.
33          // The redemption will fail if it does not contain (sufficient of)
                these Circles
34          return (redemption.redemptionIds, redemption.redemptionValues, _burnIds
                , _burnValues);
35      }
```

Impact

Anyone can force the `GroupLiquidityProvider` to exchange gCRC for worthless pCRC.

gCRC will not be backed one to one for valuable pCRC. Valuable pCRC may end up stuck in the group's vault with no gCRC to back them up.

Code Snippets

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/core-members-group/CMGRedemptionHandler.sol#L140-L168

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/core-members-group/CoreMembersGroup.sol#L342-L376

Recommendation

Only collateral trusted by the CMG should be tracked by the `CMGRedemptionHandler`. Upon adding an active ID, the redemption handler should check that `hub.isTrusted(cmGroup, ID)`.

The CMG should not allow redeeming gCRC for untrusted CRC. In `beforeRedeemPolicy()` it should enforce that the outgoing tokens are trusted by itself.

Optionally, the vault of the group should refuse to receive tokens it does not trust.

[Low] Anyone can empty tracking of collateral in CMGRedemptionHandler

Summary

Since the external function `syncValidCollateral()` has no restriction on caller, anyone who owns enough gCRC can empty one of the underlying collateral by converting their gCRC to that pCRC before calling `syncValidCollateral([pCRC])` to remove the tracking of that pCRC. They may exchange the pCRC back into the gCRC and repeat that process for each currently tracked collateral so that no collateral is tracked anymore.

Vulnerability Detail

The function `syncValidCollateral()` checks the balance of the vault for the collateralIds and remove collateral with low balances:

```
    function syncValidCollateral(uint256[] memory _collateralIds) external {
        address vault = _getGroupVault();

        // expand addresses array for batch balance check
        address[] memory accounts = new address[](_collateralIds.length);
        for (uint256 i = 0; i < _collateralIds.length; i++) {
            accounts[i] = vault;
        }

        // get balances for all IDs in single call
        uint256[] memory balances = circlesCore.hub.balanceOfBatch(accounts,
            _collateralIds);

        // update tracking for each ID based on balance
        for (uint256 i = 0; i < _collateralIds.length; i++) {
            uint256 id = _collateralIds[i];
            uint256 balance = balances[i];

            // add to active tracking if has balance above minimal amount but
                not tracked
            if (balance > minimalTrackingAmount) {
                // no-op if already tracked
                _addActiveId(id);
            }
            // remove from active tracking if has balance below minimal amount
                but is tracked
            else if (balance <= minimalTrackingAmount) {
                // no-op if id not tracked - should not occur
                _removeActiveId(id);
            }
        }
    }
```

Anyone who possesses gCRC can redeem them for one of the collateral through `StandardTreasury.onERC1155Received()`:

```
    function onERC1155Received(address _operator, address _from, uint256 _id,
        uint256 _value, bytes calldata _data)
        public
        override
        onlyHub(0xE8)
        returns (bytes4)
    {
        (bytes32 metadataType, address group, bytes memory userData) =
            _decodeMetadataForGroup(_data);
        if (metadataType == METADATATYPE_GROUPMINT) {
            ...
        } else if (metadataType == METADATATYPE_GROUPREDEEM) {
            return _redeemGroupCircles(_operator, _from, _id, _value, userData)
                ;
        } ...
    }

    function _redeemGroupCircles(address _operator, address _from, uint256 _id,
         uint256 _value, bytes memory _userData)
        internal
        returns (bytes4)
    {
        ...

        // query the mint policy for the redemption values
        uint256[] memory redemptionIds;
        uint256[] memory redemptionValues;
        uint256[] memory burnIds;
        uint256[] memory burnValues;
        (redemptionIds, redemptionValues, burnIds, burnValues) =
            policy.beforeRedeemPolicy(_operator, _from, group, _value,
                _userData);
        ...

        // return collateral Circles to the redeemer of group Circles
        vault.returnCollateral(_from, redemptionIds, redemptionValues,
            _userData);
        ....
    }
```

Anyone may be able to remove all the collateral available on the vault for one ID, and call `syncValidCollateral()` for that ID to remove its tracking before putting back the collateral.

Impact

This is a griefing attack that disables utility contracts `CMGRedemptionHandler` and `GroupLiquidityProvider` which relies on it. It can easily be solved by any honest caller of `syncValidCollateral()`

Code Snippets

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/core-members-group/CMGRedemptionHandler.sol#L140-L168

Recommendation

I am not sure how often the `syncValidCollateral()` function needs to be called. I think it's only necessary if the `CMGRedemptionHandler` has been disabled for some time on the CMG and due to not tracking low amounts of collateral. If that's the case, it could be made `onlyOwner`.

Otherwise acknowledge the issue.

[Low] CMGRedemptionHandler may be incorrect when adding / removing a collateral twice

Summary

If the functions `registerDeposit()` or `registerRedemption()` are called with twice the same collateral ID in the input array, the adding / removing of collateral may be incorrect with regards to minimal tracking amounts.

Vulnerability Detail

The function `registerDeposit()` can be called twice with for example `registerDeposit([IdA], [0.5 * minimalTrackingAmount])` which would be equivalent to a deposit of `minimalTrackingAmount` but not result in adding the `IdA` as an active ID:

```
    function registerDeposit(uint256[] memory collateralIds, uint256[] memory
        amounts) external onlyCMGroup {
        if (collateralIds.length != amounts.length) {
            revert CMGHandlerInvalidCallingParameters();
        }
        for (uint256 i = 0; i < collateralIds.length; i++) {
            uint256 id = collateralIds[i];
            if (amounts[i] > minimalTrackingAmount) {
                // no-op if already tracked
                _addActiveId(id);
            }
        }
    }
```

The function `registerRedemption()` can be called with for example `registerRedemption([IdA, IdA], [value, value])` so that the initial balance of the vault is equal to `2 * value` and `value > minimalTrackingAmount`:

```
    function registerRedemption(uint256[] memory _collateralIds, uint256[]
        memory _amounts) external onlyCMGroup {
        // CM group always registers with standard treasury
        address vault = _getGroupVault();

        // to do a batched balance call of vault for each,
        // we need to expand the address - it's a trade-off
        address[] memory accounts = new address[](_collateralIds.length);
        for (uint256 i = 0; i < _collateralIds.length; i++) {
            accounts[i] = vault;
        }

        uint256[] memory balances = circlesCore.hub.balanceOfBatch(accounts,
            _collateralIds);

```

```
        for (uint256 i = 0; i < _collateralIds.length; i++) {
            uint256 id = _collateralIds[i];

            if (balances[i] < _amounts[i]) {
                revert CMGHandlerEarlyRevertCollateralNotPresent();
            }
            uint256 remainingBalance;
            unchecked {
                // explicitly checked above, and this handler can't effect
                // the actual redemption flow by hub,
                // nor is this remaining balance stored
                remainingBalance = balances[i] - _amounts[i];
            }

            // When a collateral's balance falls below minimal tracking amount
            // we remove it from our active tracking lists
            if (remainingBalance <= minimalTrackingAmount) {
                // no-op if id not tracked -- should not occur
                _removeActiveId(id);
            }
        }
        ...
    }
```

Since the initial balance in `IdA` is computed once before loop execution and not subtracted to twice, the `remainingBalance` computed will be `initialBalance - value = value` instead of `0` and the collateral will keep on being tracked while it should be removed.

Impact

Tracking of collateral by `CMGRedemptionHandler` may be incorrect under certain conditions.

Code Snippets

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/core-members-group/CMGRedemptionHandler.sol#L75-L136

Recommendation

Acknowledge the issue.

[Info] Trust and collateral tracking may be incorrect when turned off and on again

Summary

The CMG calls the optional `CMGMintHandler` and `CMGRedemptionHandler` to mimic its trust and deposited/redeemed collateral. If they are turned off when a trust or deposit/redeem occur, they will not be called and will have incorrect state when turned back on again.

It seems developers are aware of this issue as there are `syncTrust()` and `syncValidCollateral()` functions available on the handlers to correct the state if that occurs.

Vulnerability Detail

The functions `beforeMintPolicy()` and `beforeRedeemPolicy()` optionally register the deposit / redemption by calling the redemption handler if connected:

```
    function beforeMintPolicy(
        ...
    ) external override onlyHub returns (bool) {
        ...
        if (redemptionHandlerConnected) {
            _registerDeposit(_collateral, _amounts);
        }
        return true;
    }

    function beforeRedeemPolicy(
        ...
    )
        external
        override
        onlyHubOrTreasury
        returns (
            ...
        )
    {
        ...

        // register redemption with redemption handler
        if (redemptionHandlerConnected) {
            _registerRedemption(redemption.redemptionIds, redemption.
                redemptionValues);
        }

        ...
    }
```

The function `_trust()` only mimics trust if the mint handler is connected:

```
    function _trust(address _trustReceiver, uint96 _expiry) internal {
```

```
        state.hub.trust(_trustReceiver, _expiry);
        address mintHandler_ = state.mintHandler;
        if (mintHandler_ != address(0) && mintHandlerConnected) {
            ICMGMintHandler(mintHandler_).mirrorTrust(_trustReceiver, _expiry);
        }
    }
```

The handlers can be turned off and on again by the owner of CMG:

```
    function connectMintHandler(bool _connected) external onlyOwner {
        mintHandlerConnected = _connected;
        emit HandlerConnected(HandlerType.MINT, state.mintHandler, _connected);
    }

    function connectRedemptionHandler(bool _connected) external onlyOwner {
        redemptionHandlerConnected = _connected;
        emit HandlerConnected(HandlerType.REDEMPTION, state.redemptionHandler,
            _connected);
    }
```

Impact

The trust / collateral state tracked by the handlers will be incorrect if they are not called part of the time. This can be solved by calling the `syncTrust()` and `syncValidCollateral()` functions on the handlers.

Code Snippets

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/core-members-group/CoreMembersGroup.sol#L331

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/core-members-group/CoreMembersGroup.sol#L369

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/core-members-group/CoreMembersGroup.sol#L520

Recommendation

Acknowledge the issue.

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments.

CMGRedemptionOperator does not need to be a standalone contract

I do not currently see a reason for `CMGRedemptionOperator` to be an independent contract from `GroupLiquidityProvider`. It does not help readability especially with a name that close to `CMGRedemptionHandler`. The two contracts could be merged or `CMGRedemptionOperator` could be turned into a library.

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/redemption-operator/CMGRedemptionOperator.sol

CoreMembersGroup `OwnerSet()` event in unexpected place

The contract `CoreMembersGroup` emits `OwnerSet(_owner)` in its constructor and in the external `setOwner()` function. It would be more coherent if it emitted `OwnerSet()` in the internal `_setOwner()` function similarly to the other internal functions `_setService()`, `_setMintHandler()`, `_setRedemptionHandler()`, etc ...

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/core-members-group/CoreMembersGroup.sol#L452-L454

CMGroupFactory deploys redemption operator and liquidity provider for no obvious reason

The contract `CMGroupFactory` deploys `CMGRedemptionOperator` and `GroupLiquidityProviderDeploy` in its constructor and do not use them for anything, it seems to be randomly put there.

GroupLiquidityProvider incorrect owner documentation

GroupLiquidityProvider states transferring ownership to group on deployment but does not.

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/liquidity-provider/GroupLiquidityProvider.sol#L13

Unused contract BaseRedemptionEncoder

The contract `BaseRedemptionEncoder` is unused and only exposes a function already available in `CMGRedemptionHandler`.

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/redemption-operator/BaseRedemptionEncoder.sol

numActive cannot be 0

In `CMGRedemptionHandler.findCollateralWithCursor()`, `numActive` is compared to 0 line 279 but it cannot be 0 otherwise the function would have returned earlier line 239.

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/core-members-group/CMGRedemptionHandler.sol#L279

CMGRedemptionHandler._removeActiveId() may get cursor out of range

The function `CMGRedemptionHandler._removeActiveId()` updates the cursor if it is greater than the removed id. If the last id in the active list is removed and the cursor is equal to that id, it will remain equal to that id and point outside the active id array. The check should be:

```
        if (cursor >= idx) {
            cursor--;
        }
```

(or ensure the cursor is lower than the length of the active collateral array)

https://github.com/aboutcircles/circles-groups/blob/62308de435ddf806d8ed6854b8a46beb813ba148/src/core-members-group/CMGRedemptionHandler.sol#L360